



4.2. Управление схемами баз данных

Привет! На связи шпаргалка урока 4.2. Продолжение знакомства с SQL. В ней вы найдете еще больше информации по составлению SQL-запросов.

На прошлых уроках вы уже познакомились с базами данных, научились составлять простые запросы на языке SQL. В этой шпаргалке вы откроете для себя еще больше возможностей языка SQL и найдете информацию о том, как создавать и удалять таблицы и как их изменять. Мы поговорим о типах данных, о том, как правильно выстраивать архитектуру проекта и как составлять запросы для поиска по нескольким таблицам.

Изменение таблицы

При создании база данных пустая, она не содержит ни одной таблицы. Как вы помните, все данные находятся в таблицах. До этого момента мы создавали модели в нашем приложении Spring Boot и просто запускали его. При этом в нашей БД автоматически создавались таблицы. Но мы можем и сами создавать их, ведь у нас есть замечательный язык SQL. Он позволяет выполнять гораздо больше манипуляций с данными, чем наше приложение.

Существует команда для создания таблицы.

```
CREATE TABLE table_name ( column1 datatype, column2 datatype, ... );
```

Команда начинается с ключевых слов CREATE TABLE, после которых расположено название таблицы. Также в запросе присутствует список полей с типами. Этой командой мы можем создать таблицу с колонками.

А что если мы не знаем, какие колонки нам понадобятся в будущем, и создадим пустую таблицу? Как нам добавить колонки в пустую таблицу? Для этого используют следующую команду:

```
ALTER TABLE table_name ADD column_name datatype;
```

Заметьте, что для добавления новой колонки необходимо указать еще и ее тип.

А теперь представим противоположную ситуацию. Например, мы создали таблицу с колонками и потом поняли, что некоторые колонки нам не нужны. Чтобы удалить колонку, используют следующую команду:

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Всё та же команда ALTER TABLE, но уже в сочетании с DROP COLUMN и именем колонки.

А теперь представьте ситуацию: в таблице есть колонка с данными, изначально мы назвали ее некорректно, а теперь хотим изменить имя. Мы, конечно, можем удалить ее и создать новую с необходимым названием, но тогда потеряем все данные. В таких случаях используют команду для переименования колонки:

```
ALTER TABLE table_name RENAME COLUMN column_name TO new_column_name;
```

Как вы помните, у колонки есть не только название, но и конкретный тип, который тоже можно изменять. Для этого используют следующую команду:

```
ALTER TABLE table_name ALTER COLUMN column_name TYPE new_datatype;
```

Всё просто. Теперь вы умеете создавать, удалять и изменять таблицы.

Раз уж мы остановились на изменении типов данных, давайте эту тему и продолжим.

Типы данных

При создании таблицы для всех ее столбцов необходимо указать тип данных. Он определяет диапазон значений, которые могут храниться в столбце, и их объем в памяти. PostgreSQL поддерживает большое количество различных типов данных, их условно можно разделить на подгруппы: числовые, символьные, логические, дата и время, бинарные и ряд других. Давайте рассмотрим детально каждую группу. Отдельно выделим самые популярные типы т. к. их используют в большинстве подходов при создании структуры БД.

Числовые типы данных

Тип	Описание	Диапазон	Размер
<u>SERIAL</u>	Представляет автоинкрементирующееся числовое значение. Оно образуется путем автоинкремента значения предыдущей строки. Поэтому, как правило, данный тип используется для определения идентификаторов строки	от 1 до 2147483647	4 байта
<u>SMALLSERIAL</u>	Представляет автоинкрементирующееся числовое значение. Аналог типа SERIAL для небольших чисел	от 1 до 32767	2 байта
<u>BIGSERIAL</u>	Представляет автоинкрементирующееся числовое значение. Аналог типа SERIAL для больших чисел	от 1 до 9223372036854775807	8 байт
<u>SMALLINT</u>	Хранит целочисленные значения. Имеет псевдоним int2	от -32768 до +32767	2 байта
<u>INTEGER</u>	Хранит целочисленные значения. Имеет псевдонимы int и int4	от -2147483648 до +2147483647	4 байта

 Тип	 Описание	 Диапазон	 Размер
<u>BIGINT</u>	Хранит целочисленные значения. Имеет псевдоним int8	от -9223372036854775808 до +9223372036854775807	8 байт
<u>NUMERIC</u>	Хранит числа с фиксированной точностью, которые могут иметь до 131072 знаков в целой части и до 16383 знаков после запятой		
<u>DECIMAL</u>	Хранит числа с фиксированной точностью, которые могут иметь до 131072 знаков в целой части и до 16383 знаков в дробной части. То же самое, что и NUMERIC		
<u>REAL</u>	Хранит числа с плавающей точкой. Имеет псевдоним float4	от 1E-37 до 1E+37	4 байта
<u>DOUBLE PRECISION</u>	Хранит числа с плавающей точкой. Имеет псевдоним float8	от 1E-307 до 1E+308	8 байт

Ниже приведен пример создания таблицы с несколькими типами.

```
CREATE TABLE example (
  Id SERIAL,
  TotalWeight NUMERIC(9,2),
  Age INTEGER,
  Surplus REAL
);
```

Самый используемый целочисленный тип — INTEGER, он наиболее сбалансирован с точки зрения выбора ширины диапазона, размера и быстродействия. Он по сути аналогичен такому же в Java и нужен для хранения целочисленных значений: возраста, порядкового номера, номер дома и так далее.

Если говорить о типах с произвольной точностью, то наиболее используемый здесь тип — NUMERIC, который позволяет хранить числа с очень большим количеством цифр. Он особенно рекомендуется для хранения денежных сумм и других величин, где важна точность.

Символьные типы данных

Символьные типы представлены строками с фиксированной и произвольной длиной.

Ниже приведена таблица с описанием типов.

 Тип	 Описание
<u>CHARACTER</u> (n)	Представляет строку из фиксированного количества символов. С помощью параметра задается количество символов в строке. Имеет псевдоним char(n)
<u>CHARACTER VARYING</u> (n)	Представляет строку из фиксированного количества символов. С помощью параметра задается количество символов в строке. Имеет псевдоним varchar(n)
<u>TEXT</u>	Представляет текст произвольной длины

Как вы могли заметить, VARCHAR — строка ограниченной переменной длины, а CHAR — строка фиксированной длины, дополненная пробелами. Поэтому чаще всего используют VARCHAR либо TEXT.

Логические типы данных

Логический тип представлен единственным типом BOOLEAN, принимающим одно из двух возможных значений — TRUE или FALSE. Его применяют, когда у сущности таблицы есть поле, которое содержит истинное или ложное значение. Например, в таблице с пользователями может быть колонка deleted. Если значение в этой колонке равно true, то это значит, что пользователь удалён.

```
CREATE TABLE users (  
    ...  
    deleted BOOLEAN,  
    ...  
);
```

TRUE также может быть представлен следующими строками: true, yes, on, 1. FALSE может быть представлен строками: false, no, off, 0. Регистр символов при этом не имеет значения.

Пример вставки значения TRUE для колонки с типом BOOLEAN:

```
INSERT INTO users (email_verified) VALUES ('true')

INSERT INTO users (email_verified) VALUES ('yes')

INSERT INTO users (email_verified) VALUES ('on')

INSERT INTO users (email_verified) VALUES (1)
```

Пример вставки значения FALSE для колонки с типом BOOLEAN:

```
INSERT INTO users (email_verified) VALUES ('false')

INSERT INTO users (email_verified) VALUES ('no')

INSERT INTO users (email_verified) VALUES ('off')

INSERT INTO users (email_verified) VALUES (0)
```

Типы даты и времени

Даты в БД могут храниться как с временем, так и без него. Типы с временем могут содержать информацию о часовых поясах. Давайте рассмотрим подробнее эти типы.

 Тип	 Описание	 Размер
<u>TIMESTAMP</u>	Хранит дату и время	8 байт
<u>TIMESTAMP WITH TIME ZONE</u>	То же самое, что и TIMESTAMP, только добавляет данные о часовом поясе	
<u>DATE</u>	Хранит только дату	4 байта
<u>TIME</u>	Хранит время с точностью до 1 микросекунды без указания часового пояса	8 байт
<u>TIME WITH TIME ZONE</u>	Хранит время с точностью до 1 микросекунды с указанием часового пояса	12 байт

TIMESTAMP применяется, например, для сохранения даты и времени регистрации, DATE — для даты рождения, TIME — для хранения желаемого времени получения рассылок email. Пример создание таблицы:

```
CREATE TABLE users (  
    ...,  
    sign_up_date TIMESTAMP,  
    birth_date DATE,  
    notification_send_time TIME,  
    ...  
);
```

Прочие типы данных

Есть еще ряд типов, которые могут вам показаться интересными.

Для работы с денежными единицами определен тип MONEY, который может принимать значения в диапазоне от -92233720368547758.08 до +92233720368547758.07 и занимает 8 байт.

А вот для хранения бинарных данных определен тип BINARY. Он хранит данные в виде бинарных строк, которые представляют последовательность октетов или байт. Этот тип можно использовать для хранения файлов в БД.

Также интересны типы для хранения геометрических типов. Например, тип POINT представляет точку на плоскости в формате (x,y) и занимает 16 байт. А если необходимо хранить информацию о полигоне (т. е. наборе точек), то следует использовать тип POLYGON. Он представляет многоугольник в формате ((x1,y1),...) и занимает 40+16n байт.

И последний тип, который мы рассмотрим, — это JSON. Он хранит данные JSON в текстовом виде. При этом можно составлять запросы для поиска по ключам или значениям.

Ограничения

Как вы могли заметить, у различных типов данных различные ограничения. Например, у целочисленных типов есть максимальные и минимальные значения. Или, например, в ячейку с типом BOOLEAN нельзя вставить строку.

Эти ограничения заложены в сами типы. Но что если нам нужны свои ограничения? Например, мы хотим создать пользователя. При этом его имя не должно быть пустым, что логично. Или возраст пользователя не должен быть меньше нуля. Для такого рода проверок SQL предоставляет нам подобный механизм ограничений значений в колонках таблиц, который позволяет задавать произвольные ограничения.

При попытке вставки данных в таблицу, нарушающих наши условия, возникнет ошибка, и данные не сохранятся, оставив таким образом правильность (или, как еще говорят, консистентность) данных.



Консистентность — соответствие имеющейся в базе данных информации ее внутренней логике, структуре и всем явно заданным правилам.

К основным ограничениям можно отнести следующие: ограничения-проверки, ограничения на NULL-значения, ограничения на уникальность значения.

Ограничения-проверки

Это наиболее универсальный тип ограничений. Он позволяет при вставке значения в таблицу проверять некоторое логическое выражение на истинность. Например, возраст пользователя можно ограничить положительными значениями так:

```
CREATE TABLE users (  
    ...,  
    age INTEGER CHECK (age > 0),  
    ...  
);
```

Ограничение определяется после типа данных. Для этого указывается ключевое слово CHECK и сама проверка в скобках. Также должно быть указано название колонки, для которой создается ограничение, иначе эта операция становится бессмысленной.

На практике так бывает, что при создании таблицы не всегда получается продумать все мелочи и приходится добавлять ограничения после создания таблицы. Добавить ограничение можно следующей командой:

```
ALTER TABLE users  
  
ADD CONSTRAINT age_constraint CHECK (age > 0);
```

Обратите внимание, что у ограничения есть название. В нашем случае это `age_constraint`. Имея эту информацию, можно удалить ограничение по имени. Для этого используется следующая команда:

```
ALTER TABLE users DROP CONSTRAINT age_constraint;
```

Ограничения на NULL-значения

Ограничения NOT NULL проверяют, что поле не содержит NULL. NULL означает, что поле не содержит значения. Однако поле и пустая строка ("") не являются пустыми значениями.

Ниже представлен пример того, как можно добавить ограничение на поле `name`. Оно запретит создавать пользователя с пустым именем.

```
CREATE TABLE users (  
    ...,  
    name TEXT NOT NULL,  
    ...  
);
```

Также можно добавить это ограничение уже после создания таблицы.

```
ALTER TABLE users  
ALTER COLUMN name SET NOT NULL;
```

Ограничения на уникальность значения

Ограничения уникальности говорят о том, что значения в колонке уникальны среди всех строк в таблице. Предположим, что у нас стоит задача сделать так, чтобы никнеймы у всех пользователей нашего сервиса были уникальны. Сделать это можно следующим запросом:

```
CREATE TABLE users (  
    ...,  
    nickname TEXT UNIQUE,  
    ...  
);
```

Или этим запросом:

```
ALTER TABLE users  
ADD CONSTRAINT nickname_unique UNIQUE (nickname);
```

Также можно добавить проверку уникальности по нескольким колонкам. Например, мы хотим запретить создавать пользователей с одинаковыми парами значений для логина и пароля. Сделать это можно следующей командой:

```
ALTER TABLE users  
ADD CONSTRAINT login_pass_unique UNIQUE (login, password);
```



У команды UNIQUE нет ограничений на количество колонок.

Остальные ограничения

Язык SQL позволяет нам комбинировать несколько проверок для одной колонки. Например, мы хотим сделать поле login уникальным и не равным нулю. Для этого мы можем использовать следующую команду:

```
CREATE TABLE users (  
    ...,  
    login TEXT UNIQUE NOT NULL,  
    ...  
);
```

Но есть другая команда, которая группирует в себе эти две проверки. Называется такая команда первичным ключом. Если говорить простым языком, то это столбец (или группа столбцов), которые могут выступать уникальным идентификатором строк таблицы. Чтобы создать первичный ключ, используется следующая команда:

```
CREATE TABLE users (  
    ...  
    login TEXT PRIMARY KEY,  
    ...  
);
```

Либо эта команда:

```
ALTER TABLE users  
ADD PRIMARY KEY (login);
```

О первичных ключах мы еще будем говорить в следующих главах.

И последний тип ограничений, которые мы сегодня рассмотрим, — это значения по умолчанию. Оно добавляется в момент, когда происходит вставка записи и для колонки не передаётся значение. Если значение по умолчанию не указано явно, то считается, что такое значение — NULL. Посмотрим на примере указания значения аватара по умолчанию для пользователя, не указавшего его при создании профиля.

```
CREATE TABLE users (  
    ...,  
    profile_picture_url TEXT DEFAULT 'https://default_picture_url',  
    ...  
);
```

Первичный ключ

Раз уж мы начали говорить о первичном ключе, давайте разберемся с ним окончательно.

Первичный ключ (primary key) применяется для уникальной идентификации записей таблицы. Никакие из двух записей таблицы не могут иметь одинаковых значений первичного ключа. Первичный ключ обычно сокращенно обозначают как РК (primary key).

В реляционных базах данных практически всегда разные таблицы логически связаны друг с другом. Первичные ключи как раз используются для однозначной организации такой связи.

Давайте рассмотрим пример. У нас есть таблица с работниками users:

id	name	department_id
1	Nikolay	1
2	Maxim	1
3	Bob	2

И таблица с подразделениями departments:

id	name
1	Marketing
2	Development
3	Support

Как вы могли заметить, у таблицы с подразделениями есть идентификатор (id), он же первичный ключ. Он однозначно идентифицирует подразделение. Мы используем идентификатор в таблице с сотрудниками. Таким образом мы связали две этих таблицы.



Первичный ключ — это поле, которое используется для обеспечения уникальности данных в таблице.

Это означает, что значение (информация) в поле первичного ключа в каждой строке (записи) таблицы может быть уникальным. Например, для создания таблицы сотрудников применяется следующая команда:

```
CREATE TABLE users (  
  id REAL,
```

```
name TEXT PRIMARY KEY,  
department_id TEXT REFERENCES departments (id)  
)
```

Как видите, у таблицы сотрудников тоже есть первичный ключ, и мы его указываем командой PRIMARY KEY. А также командой REFERENCES указываем на первичный ключ таблицы подразделения.

Нормализация

Мы уже изучили достаточно много SQL-команд, чтобы манипулировать структурой БД. Настало время поговорить про нормальные формы. Соблюдение этих правил приведет к отсутствию избыточности функциональных зависимостей между атрибутами.



Нормальная форма (НФ) — требование, предъявляемое к структуре таблиц в теории реляционных баз данных для устранения из базы избыточных функциональных зависимостей между атрибутами (полями таблиц).

Процесс проектирования БД с использованием метода НФ является итерационным и заключается в последовательном переводе отношения из 1НФ в НФ более высокого порядка по определенным правилам. Каждая следующая НФ ограничивается определенным типом функциональных зависимостей и устранением соответствующих аномалий при выполнении операций над отношениями БД, а также сохранении свойств, предшествующих НФ.

Первая нормальная форма

Отношение находится в 1НФ, если все его поля являются простыми, при этом все используемые множества допустимых значений должны содержать только скалярные значения. Не должно быть повторений строк в таблице.

Например, есть таблица с пользователями и их телефонами:

 username	 phone_number
<u>Alice</u>	<u>+79995553535,+79991112233</u>
<u>Bob</u>	<u>+79994567890</u>

Нарушение нормализации 1НФ происходит в строке пользователя Alice, т. к. в одной ячейке содержится список из двух телефонов: +79995553535, +79991112233, т. е. список не является атомарным. Преобразуем таблицу к 1НФ:

 username	 phone_number
<u>Alice</u>	<u>+79995553535</u>
<u>Alice</u>	<u>+79991112233</u>
<u>Bob</u>	<u>+79994567890</u>

Теперь каждая ячейка содержит атомарное значение.

Вторая нормальная форма

Перейдем ко второй нормальной форме. Представим, что у нас есть таблица users, в которой мы храним пользователей, их роли и возможность выполнять те или иные действия в зависимости от роли.

 username	 role	 permissions
<u>Alice</u>	admin	can_edit
<u>Bob</u>	user	can_write
<u>Carol</u>	user	can_write

Можно увидеть, что наша таблица users находится в первой нормальной форме. Также можно заметить, что permissions зависит от role.

Требования ко второй нормальной форме гласят, что значение в каждой колонке должно зависеть от ключа и только от ключа, которым в нашем случае выступает значение в колонке username.



Также не забываем, что отношение 2НФ тоже должно соблюдать правила 1НФ.

Но в нашей таблице это не так — значение в колонке permissions зависит от role. Значит, наша таблица не находится во второй нормальной форме. Для того чтобы это исправить, нам нужно устранить эту зависимость и вынести ее в отдельную таблицу, где ключом будет выступать role. Проводя процедуру нормализации, получим две таблицы: Users и Roles.

Users (1)

<u>Aa</u> username	role
<u>Alice</u>	admin
<u>Bob</u>	user
<u>Carol</u>	user

Roles (1)

<u>Aa</u> role	permissions
<u>admin</u>	can_edit
<u>user</u>	can_write

Таким образом, мы получили таблицы, значения в которых зависят только от ключа.

Третья нормальная форма

Требование третьей нормальной формы заключается в том, чтобы в таблицах отсутствовала транзитивная зависимость.



Транзитивная зависимость — это когда неключевые столбцы зависят от значений других неключевых столбцов.

Если в первой нормальной форме мы пытались соблюсти реляционные принципы, во второй нормальной форме мы выделяли первичный ключ, то в третьей нормальной форме всё наше внимание уделено столбцам, которые не являются первичным ключом, т. е. неключевым столбцам.

Для нормализации базы данных до третьей нормальной формы необходимо сделать так, чтобы в таблицах отсутствовали неключевые столбцы, которые зависят от других неключевых столбцов.

Иными словами, неключевые столбцы не должны пытаться играть роль ключа в таблице, т. е. они действительно должны быть неключевыми столбцами. Такие столбцы не дают возможность получить данные из других столбцов, они дают возможность посмотреть на информацию, которая в них содержится, так как в этом их назначение.

Рассмотрим пример с сотрудниками:

# id	<u>Aa</u> name	 position	 department	 description
1	<u>Bob</u>	Developer	Development	Разработка и сопровождение приложений и сайтов
2	<u>Sam</u>	Manager	Management	Управление разработкой
3	<u>John Smith</u>	QA	Testing	Тестирование продукта

Давайте рассмотрим этот пример и попытаемся выяснить, находится ли эта таблица в третьей нормальной форме? Проверим все неключевые столбцы, каждый из них должен зависеть только от первичного ключа, и никаким образом к другим неключевым столбцам он не должен относиться.

Несложно заметить, что столбец description не зависит напрямую от первичного ключа. Мы это выяснили, когда задали себе один вопрос: «Каким образом описание подразделения связано с сотрудником?». И можем сделать следующее заключение: атрибут «описание подразделения» содержит детальные сведения того подразделения, в котором работает сотрудник.

Отсюда следует, что столбец `description` не связан напрямую с сотрудником, он связан напрямую со столбцом `department`, который напрямую связан с сотрудником, ведь сотрудник работает в каком-то конкретном подразделении. Это и есть транзитивная зависимость, когда один неключевой столбец связан с первичным ключом через другой неключевой столбец.

Как думаете, что мы должны сделать, чтобы привести эту таблицу к третьей нормальной форме? Правильно, декомпозицию!

Мы должны эту таблицу разбить на две: в первой хранить сотрудников, а во второй — подразделения. А для реализации связи в таблице сотрудников нужно создать ссылку на таблицу подразделений, т. е. добавить внешний ключ.

Users (1)

# id	Aa name	≡ position	# department_id
1	<u>Bob</u>	Developer	1
2	<u>Sam</u>	Manager	2
3	<u>John Smith</u>	QA	3

Таблица подразделений в третьей нормальной форме.

Departments (1)

# id	Aa department	≡ description
1	<u>Development</u>	Разработка и сопровождение приложений и сайтов
2	<u>Management</u>	Управление разработкой
3	<u>Testing</u>	Тестирование продукта

Таким образом, в наших таблицах отсутствует транзитивная зависимость и они находятся в третьей нормальной форме.

Работа с несколькими таблицами

И напоследок разберем, как в одном запросе получить данные с нескольких таблиц. Рассмотрим на предыдущем примере. Мы разделили сотрудников и подразделения на отдельные таблицы. Для того чтобы получить всю информацию

о них, в одном запросе будем использовать JOIN в SQL. JOIN соединяет данные из двух или более таблиц на основе соотносящихся колонок. Существует несколько видов JOIN, отличающихся нюансами их работы — INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN и прочие.

Посмотрим еще раз на таблицы сотрудников и подразделений.

User

id	name	position	department_id
1	Bob	Developer	1
2	Sam	Manager	2
3	John Smith	QA	3
4	Bilbo	Support	

Таблица подразделений в третьей нормальной форме.

Departments

id	department	description
1	Development	Разработка и сопровождение приложений и сайтов
2	Management	Управление разработкой
3	Testing	Тестирование продукта
4	Sales	Продажа продукта

Представим теперь, что мы хотим увидеть таблицу в исходном виде:

name	position	department	description
Bob	Developer	Development	Разработка и сопровождение приложений и сайтов
Sam	Manager	Management	Управление разработкой
John Smith	QA	Testing	Тестирование продукта

Подумаем, что нам для этого потребовалось бы? Нам нужно найти все строки во второй таблице, которые имеют ключ, соответствующий значению в колонке department_id в первой таблице. Давайте визуализируем это рассуждение, раскрасив одинаковыми цветами строки в двух таблицах, имеющих совпадающие значения в колонке role.

User

id	name	position	department_id
1	Bob	Developer	1
2	Sam	Manager	2
3	John Smith	QA	3
4	Bilbo	Support	

Departments

id	department	description
1	Development	Разработка и сопровождение приложений и сайтов
2	Management	Управление разработкой
3	Testing	Тестирование продукта
4	Sales	Продажа продукта

Давайте теперь напишем запрос с использованием JOIN, чтобы получить желаемый результат в виде исходной таблицы до разделения.

```
SELECT users.name, users.position, department.department, department.description
FROM users
INNER JOIN department ON users.department_id = department.id
```

После ключевых слов INNER JOIN указывается название таблицы, после ключевого слова ON — логическое выражение, по которому определяется, должны ли быть объединены строки таблицы. В SELECT необходимо указать, помимо названий колонок, которые нужно вывести в ответе, название таблиц, следующих за INNER JOIN, из которых нужно вывести эти колонки.

Результатом выполнения этого запроса будет следующий ответ:

name	position	department	description
Bob	Developer	Development	Разработка и сопровождение приложений и сайтов
Sam	Manager	Management	Управление разработкой
John Smith	QA	Testing	Тестирование продукта

Заметьте, что мы использовали INNER JOIN. Он необходим для получения только тех строк, для которых существует соответствие записей главной таблицы и присоединяемой. Иными словами, условие должно выполняться всегда.

А теперь давайте составим запрос для LEFT JOIN.

```
SELECT users.name, users.position, department.department, department.description
FROM users
LEFT JOIN department ON users.department_id = department.id
```

И получим следующее:

name	position	department	description
Bob	Developer	Development	Разработка и сопровождение приложений и сайтов
Sam	Manager	Management	Управление разработкой
John Smith	QA	Testing	Тестирование продукта
Bilbo	Support	null	null

В случае с LEFT JOIN из главной таблицы будут выбраны все записи, даже если в присоединяемой таблице нет совпадений, то есть условие не учитывает присоединяемую (правую) таблицу. Результат показывает всех сотрудников и их подразделения, вне зависимости от того, есть они или нет.

RIGHT JOIN действует наоборот: возвращает все значения из правой таблицы и значения из левой, если они существуют.

```
SELECT users.name, users.position, department.department, department.description
FROM users
RIGHT JOIN department ON users.department_id = department.id
```

Результат такой команды будет следующий:

name	position	department	description
Bob	Developer	Development	Разработка и сопровождение приложений и сайтов
Sam	Manager	Management	Управление разработкой
John Smith	QA	Testing	Тестирование продукта
null	null	Sales	Продажа продукта

А вот FULL OUTER JOIN (ключевое слово OUTER можно опустить) необходим для отображения всех возможных комбинаций строк из нескольких таблиц. Иными словами, это объединение результатов LEFT и RIGHT JOIN.

```
SELECT users.name, users.position, department.department, department.description
FROM users
FULL JOIN department ON users.department_id = department.id
```

А результат будет таким:

name	position	department	description
Bob	Developer	Development	Разработка и сопровождение приложений и сайтов
Sam	Manager	Management	Управление разработкой
John Smith	QA	Testing	Тестирование продукта
null	null	Sales	Продажа продукта
Bilbo	Support	null	null